ENME 441

Peer-to-Peer Communication

`socket` → `HTTP` → `requests`

Topics

PART 1

`socket`

Raw TCP endpoints

- bind / listen / accept
- Client and server roles
- Bytes on the wire

PART 2

`HTTP`

Serving a web page

- Request & response format
- GET vs POST
- HTML forms for control

PART 3

`requests`

Calling out to the web

- The board as a client
- JSON payloads over TLS
- Webhooks and REST APIs

The Network Matters

1

umd-iot blocks all inbound connections

An ESP32 web server on umd-iot is unreachable from other devices on umd-iot, eduroam, or anywhere else

2

Peer-to-peer communication requires a network that permits peer traffic

A home router, a lab network, or a phone hotspot. Both devices must sit on the same subnet with client isolation turned off.

3

Add your cell phone hotspot to config.py

WIFI_NETWORKS is a list of (ssid, password) pairs and wifi.py walks it in order — so the same board works in both places with no code change.

4

Two architectures, two situations

When you don’t control the network, let a broker relay for you (brokered communication).

01 socket

Raw TCP endpoints

Sockets

A socket is one end of a two-way link between two programs on a network.

1

An endpoint, identified by address + port

The IP address specifies the machine; the port specifies which program on the machine handles the communications.

2

The API has barely changed since 1971

Berkeley (BSD) sockets date back to ARPANET. Python's socket module is a thin wrapper over the same C calls used everywhere else.

3

We use TCP over IPv4

AF_INET selects IPv4 addressing, SOCK_STREAM selects TCP — which guarantees your bytes arrive, in order, exactly once.

4

Everything else is built on this

HTTP, MQTT, SSH and the web itself are just agreed-upon message formats pushed through an ordinary socket.

Two Roles, Two Sets of Calls

The server waits to be found; the client goes looking.

SERVER

waits for connections

- **`socket()`**
  create the socket
- **`bind()`**
  claim an IP address and port
- **`listen()`**
  start queueing connection requests
- **`accept()`**
  take the next client — blocks
- **`recv()` / `send()` / `sendall()`**
  exchange data on that connection
- **`close()`**
  release the connection

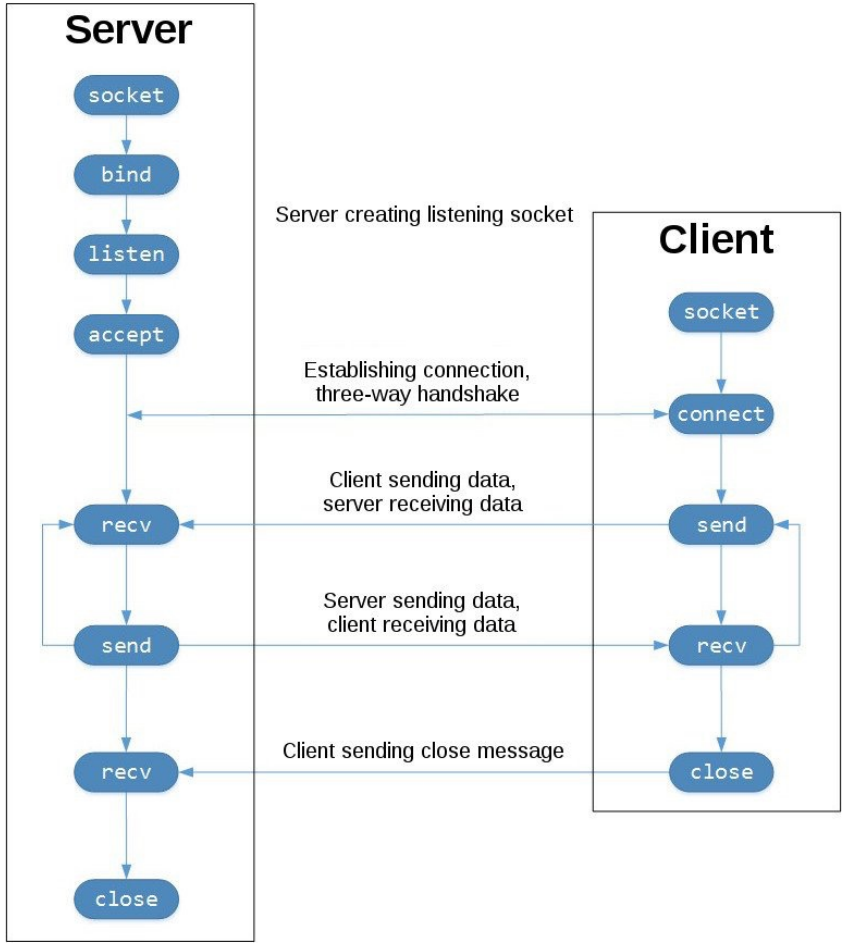
CLIENT

initiates the connection

- **`socket()`**
  create the socket
- **`connect()`**
  ask a server to accept
- **`send()` / `sendall()` / `recv()`**
  exchange data
- **`close()`**
  release the connection

The Connection Lifecycle

Who calls what, and in which order.



- **The server sets up first**

  socket → bind → listen → accept. The serve must be waiting before any client can reach it.

- **accept() blocks until someone arrives**

  Execution stops on accept(). Nothing else in that thread runs until a client connects.

- **Then data flows both ways**

  Any number of send/recv rounds ride on the same open connection — no reconnecting between messages.

- **Always close**

  Both ends should agree the exchange is finished. New connections can always be made to for ongoing communication.

`send()` vs. `sendall()`

sendall() is send() called in a loop. The difference only shows up when the socket will not take everything at once.

`send()` the primitive — you own the leftovers

```
n = conn.send(data)          # n may be < len(data)
sent = 0
while sent < len(data):      # loop, or lose bytes
    sent += conn.send(data[sent:])
```

- **Returns a count**
  The number of bytes the kernel accepted into the send buffer — not necessarily all of them.
- **A short write is normal**
  When the buffer fills — slow peer, large payload — the count comes back small. Discard it and you have silently truncated your data.
- **Use it when you need that count**
  Non-blocking sockets, resumable transfers, or dropping stale frames when the peer falls behind.

`sendall()` the loop, already written for you

```
conn.sendall(data)           # returns None
# in effect, it does this for you:
#     while data:
#         data = data[conn.send(data):]
```

- **All of it, or an exception**
  Returns None on success. No count to check, no partial state to carry around.
- **It blocks until the last byte is queued**
  On a blocking socket it waits for the peer to drain the buffer, so your program stops on that line.
- **It cannot say how far it got**
  If it raises part-way through, the number of bytes already sent is unknowable — fine when the next step is closing the connection anyway.

sendall() is the default for standard use

Nothing useful to do with a short count. Half an HTTP response or half a JSON message is garbage to the receiver. The only correct move is to send the rest, which is exactly what sendall() does.

It clearly states the intent. sendall() means “this whole message goes out.” send() means “send what fits, and I will manage the remainder myself.”

Note: there is no recvall(), because only your protocol knows where a message ends.

Server Side

`bind()` → `listen()` → `accept()` — claim a port, then wait for a client

```python
import socket

HOST = ''       # '' = accept on any interface
PORT = 8080     # >1023 avoids privileged ports

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Reuse the port even if a previous socket is stuck
# in TIME_WAIT (prevents an EADDRINUSE error):
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

s.bind((HOST, PORT))
s.listen(3)                     # queue up to 3 clients

conn, (addr, port) = s.accept()    # BLOCKS here
data = conn.recv(1024)             # up to 1024 bytes
conn.sendall(data)                 # echo it back
conn.close()
```

- **AF_INET, SOCK_STREAM**

  IPv4 plus TCP — the pair you will use for everything in this course.

- **SO_REUSEADDR before bind()**

  Without it, restarting your script after a crash often fails for a minute or two while the OS holds the port.

- **accept() returns a NEW socket**

  conn is the connection to that one client. The original s keeps listening for the next.

- **recv() gives you bytes**

  Not str. Decode before treating it as text, and note that 1024 is a maximum, not a guarantee.

Client Side

Far shorter — the client only has to know where to call.

```
import socket

c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect((HOST, PORT))         # ask the server to accept

c.sendall(b'hello server')      # bytes, not str
reply = c.recv(1024)
print(reply.decode('utf-8'))

c.close()
```

Every trip through a socket is bytes:

```
b'hello'                    # a bytes literal
'hello'.encode('utf-8')     # str   -> bytes
reply.decode('utf-8')       # bytes -> str
```

- **connect() is the whole handshake**

  Three packets go back and forth underneath; you just see the call return.

- **sendall() vs send()**

  send() may transmit only part of your data and return how much. sendall() loops until it is all gone — prefer it.

- **The port must match exactly**

  A client on the wrong port gets ECONNREFUSED, even with the right address.

- **UTF-8 is the safe default**

  It encodes every Unicode character, and it is what browsers and web APIs assume.

echo_server.py — Both Ends at Once

`echo_server.py` runs a server and a client in one script, so you can try sockets with one device

```
HOST, PORT = '127.0.0.1', 65432    # loopback = this device

def server():
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((HOST, PORT))
    s.listen(3)
    conn, addr = s.accept()             # blocking
    while True:
        data = conn.recv(1024).decode('utf-8')
        reply = f'Server received: {data.upper()}'
        conn.sendall(reply.encode('utf-8'))

_thread.start_new_thread(server, ())    # server gets its own thread
time.sleep(1)                           # let it reach accept()

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))
while True:
    client.sendall(b'Hello server')
    print(client.recv(1024).decode('utf-8'))
    time.sleep(3)
```

- **127.0.0.1 never leaves the device**

  The loopback address lets both halves talk without a network at all — ideal for a first experiment.

- **The server needs its own thread**

  accept() and recv() both block. Without a thread, the client code below would never be reached.

- **sleep(1) is not decoration**

  The client must not call connect() before the server reaches accept(), or it gets ECONNREFUSED.

- **Encode out, decode in**

  The upper-casing happens on a str; the socket only ever carries bytes.

Sockets: Things to Watch For

accept() and recv() block

Your program stops on that line until something arrives. If anything else has to keep running, put the server in a thread.

Bytes, never str

send(), sendall() and recv() all deal in bytes. A forgotten .encode() is the single most common socket error.

EADDRINUSE after a crash

The OS holds the port in TIME_WAIT for a minute or two. Set SO_REUSEADDR before bind(), or wait it out.

recv(n) is not a message boundary

TCP is a byte stream, not a datagram. One send() can arrive split across two recv() calls, and two sends can arrive merged into one.

02

HTTP

Serving a web page from the board

HTTP and HTML

The protocol that carries the web, and the language it usually carries.

1

HTTP is a request/response protocol

The client sends a formatted text request; the server replies with a formatted text response. That is the entire idea.

2

It rides on an ordinary TCP socket

Everything you send is plain text through a TCP connection you already know how to open — there is no magic layer underneath.

3

HTML is what the response usually carries

A markup language the browser renders as a page. w3schools.com/html is an excellent reference.

4

You are writing the server by hand

Which means producing byte-exact status lines, headers and blank lines yourself. The browser is unforgiving about all three.

The Request

Generated by the client; your server needs to parse it.

```
GET /?val1=-2.2&pin2=on HTTP/1.1
Host: 192.168.0.133
Accept: text/html,application/xhtml+xml
Accept-Encoding: gzip, deflate
```

start line · **headers** · blank line, then body (empty for this GET)

1 **Start line**

Method (GET, POST, PUT, DELETE), the target path — which may carry data after a ? — and the HTTP version.

2 **Headers**

One key: value pair per line: host, content length, accepted encodings, and so on.

3 **Blank line (\r\n\r\n), then optional body**

Headers end with \r\n\r\n. GET normally sends no body; POST puts its data in the body.

The Response

Generated by your server; the browser must be able to parse it.

```
HTTP/1.1 200 OK
Content-type: text/html
Connection: close

<html><body>hello there</body></html>
```

…which you produce one line at a time:

```
conn.send(b'HTTP/1.1 200 OK\r\n')           # status line
conn.send(b'Content-type: text/html\r\n')    # header
conn.send(b'Connection: close\r\n\r\n')      # last header + BLANK LINE
conn.sendall(b'<html>...</html>')            # body
```

- **Status line**
  HTTP version, a numeric code (below), and optional text.
  200 OK
  404 Not Found
  201 Created
- **Headers tell the browser what to do**
  Content-type: text/html tells the browser to render the message body as a web page instead of showing raw text.
- **The blank line is mandatory**
  \r\n\r\n on the LAST header only. Send it after every header and the browser sees an empty page.
- **Body is optional**
  A bare status line is a complete response — useful for a 404, or an endpoint that only acknowledges.

webserver.py — A Web Server in 20 Lines

`webserver.py` serves a page of live GPIO pin states to any browser that sends a request

```
def web_page():
    pins = [machine.Pin(n, machine.Pin.IN) for n in range(8)]
    rows = [f'<tr><td>{p}</td><td>{p.value()}</td></tr>' for p in pins]
    html = '<html><body><h1>Pins</h1><table>' + '\n'.join(rows) + '</table></body></html>'
    return bytes(html, 'utf-8')

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(('', 80))
s.listen(1)
while True:
    conn, addr = s.accept()          # blocking
    request = conn.recv(1024)        # read even if unused
    conn.send(b'HTTP/1.1 200 OK\r\n')
    conn.send(b'Content-type: text/html\r\n')
    conn.send(b'Connection: close\r\n\r\n')
    try:
        conn.sendall(web_page())
    finally:
        conn.close()
```

- **Read the request even if you ignore it**

  Skipping recv() leaves data sitting in the buffer and the browser can hang waiting to be heard out.

- **Port 80 needs no sudo here**

  There is no operating system enforcing privileged ports on the board — unlike the Pi, where port 80 required root.

- **Find the address from wifi.py**

  connect_wifi() prints wlan.ifconfig(); the first entry is what you type into your browser.

- **close() inside finally**

  A browser that vanishes mid-response should not take the whole server down with it.

Run the Server in its Own Thread or Task

accept() blocks, so run the server in its own thread and get your main loop back. Alternately, use asyncio.start_server() to use asyncio tasks if you need more control over task switching (not covered here).

```
import _thread

def serve_web_page():
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind(('', 80))
    s.listen(3)
    while True:
        conn, addr = s.accept()    # blocks THIS thread only
        request = conn.recv(1024)
        conn.send(b'HTTP/1.1 200 OK\r\n')
        conn.send(b'Content-type: text/html\r\n\r\n')
        try:
            conn.sendall(web_page())
        finally:
            conn.close()

_thread.start_new_thread(serve_web_page, ())

while True:         # main loop keeps running
    sleep(1)
    print('.')
```

- **No join(), no way to kill it**

  A MicroPython thread runs until the board resets. machine.reset() from the REPL is your stop button. Using asyncio instead of _thread can avoid this limitation.

- **Now sensors can run alongside the server**

  Read a sensor, drive a motor, blink an LED in the main thread, while the server sits waiting in the background.

- **Shared state demands locks**

  The thread and your main loop see the same globals, so remember to use locks as needed.

GET and POST

Two ways for the browser to hand data to your server.

1

GET for reads, POST for writes

Use GET when the request will not change anything on the server; POST when it will. Both work on your own server — POST just documents the intent.

2

GET data rides in the URL

After a ? in the start line: /?gpio1=on&gpio2=off. Visible in the address bar, saved in history, and length-limited.

3

POST data rides in the body

After the blank line. Not visible in the URL, and comfortable with much larger payloads.

4

Same shape either way

key=value pairs, joined with &, no spaces: val1=-2.2&val2=6&pin2=on

5

Neither one is secure

Both are plain text on the wire. Only HTTPS/TLS encrypts them — see the end of this lecture.

Where the Data Actually Sits

The same three fields, carried in two different places.

GET — data in the start line

```
GET /?val1=-2.2&val2=6&pin2=on HTTP/1.1
Host: 192.168.0.133
Accept: text/html,application/xml
Accept-Encoding: gzip, deflate
```

body is empty

POST — data in the body

```
POST / HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Accept: text/html,application/xml

val1=-2.2&val2=6&pin2=on
```

everything after the blank line

GET and POST requests are parsed differently:

GET: slice the start line between `GET` and `HTTP`
POST: locate `\r\n\r\n` and read the remaining body content

web_gpio_get.py — Control a Pin from a Browser

```
led = Pin(8, Pin.OUT)      # built-in LED, inverted logic

def web_page():
    state = 'OFF' if led.value() else 'ON'
    html = """<html><body>
      <p>LED state: <strong>""" + state + """</strong></p>
      <form action="/" method="GET">
        <button type="submit" name="button_on">ON</button>
        <button type="submit" name="button_off">OFF</button>
      </form></body></html>"""
    return html.encode('utf-8')

while True:
    conn, addr = s.accept()
    data = conn.recv(1024).decode('utf-8')
    # slice the query string out of the start line:
    data = data[data.find('GET') + 6 : data.find('HTTP')]
    if 'button_on'  in data: led.value(0)
    if 'button_off' in data: led.value(1)
    ...
```

- **Slice between GET and HTTP**
  A crude but effective way to isolate the query string from the rest of the start line.
- **Inverted LED logic**
  value(0) turns the built-in LED ON — there is a pull-up resistor on that pin.
- **decode('utf-8')**
  Values arrive URL-encoded: spaces become +, punctuation becomes %XX. Decoding ensures you can accept free text.
- **GET is the wrong method here**
  This request changes a GPIO state, so POST is the correct choice. That is the next slide.

Try it: import web_gpio_get at the end of main.py, reboot your ESP32, and check your IP address. With your computer connected to the same network as the ESP32 (your phone hotspot) enter `http://your.ip.add.ress` into a browser to load the web page from the server.

web_gpio_post.py — Parsing the Body

The simple helper function parse_post() turns the POST body into a dictionary you can index.

```
def parse_post(msg):
    data = {}
    body = msg[msg.split('\r\n\r\n', 1)[1]    # skip headers
    for pair in body.split('&'):
        if '=' in pair:
            key, value = pair.split('=', 1)
    return data

conn, addr = s.accept()
client_message = conn.recv(2048).decode('utf-8')
data = parse_post(client_message)

if 'led_byte' in data:          # was anything posted?
    led_byte = data['led_byte']
else:
    led_byte = '0'              # first load of the page
```

- **+4 skips the blank line**

  \r\n\r\n is four characters. Everything after it is the body.

- **Always check the key exists**

  The very first page load carries no POST data at all, so index blindly and you get a KeyError.

OPTIONAL

Making Your Own Requests from the Browser

GET type it in the address bar

```
// the address bar is already a GET client:
http://192.168.0.133/?button_on=1
http://192.168.0.133/?button_off=1
http://192.168.0.133/?val1=-2.2&val2=6&pin2=on

// press Enter and your serve loop sees a
// complete GET request, exactly as written
```

POST F12 / Cmd-Opt-I, then the Console tab

```
// on a page served by the board:
fetch('/', {
  method: 'POST',
  headers: {'Content-Type':
      'application/x-www-form-urlencoded'},
  body: 'led_byte=170'
})
```

- **Everything after the ? is your data**
  That is the same query string web_gpio_get.py pulls out of the start line with data[data.find('GET')+6 : data.find('HTTP')].
- **Type the http:// yourself**
  A bare 192.168.0.133 gets treated as a search term, or quietly upgraded to https (which your server does not speak).
- **Run it on the board's own page**
  Load http://192.168.0.133/ first, then open the console there. From any other page the POST still reaches the ESP32 — the LEDs change — but CORS stops JavaScript from reading the reply.
- **The Content-Type is what makes it parse**
  application/x-www-form-urlencoded is what turns the body into key=value&key=value, which is exactly what parse_post() expects.

- **Watch for**

  The Network tab shows the real message. Click a request to see the exact start line, headers, and body your board received. Firefox's "Edit and Resend" lets you change one and fire it again.

  Repeated GETs may be cached. An identical GET can be answered by the browser without ever reaching the board. Change a parameter, or hard-reload, if nothing seems to happen.

HTML Form Elements

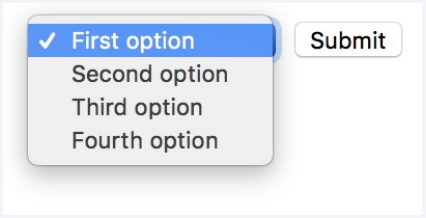
Each one becomes a key in the dictionary your parser returns — via its name attribute.

Button

```
<button type="submit">
```

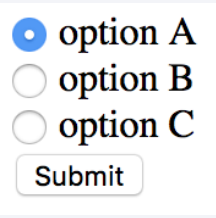
Select menu

```
<select>
```



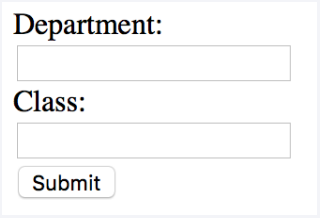
Radio buttons

```
<input type="radio">
```



Text input

```
<input type="text">
```



Range slider

```
<input type="range">
```

Many more available

checkbox, number, color, date, hidden (for values the user never sees), and so on.

w3schools.com/html/html_form_elements.asp

Scaffolding the HTML with an LLM

A prompt that produces something usable:

```
Generate an HTML form with two buttons on one row
labeled "ON" and "OFF", a range slider beneath them
labeled "Set Value", and a submit button.
Use method="POST" and action="/".
Give every input a name attribute. No CSS frameworks,
no external files -- one self-contained page.
```

…then check these four things before you serve it:

```
<form action="/" method="POST">      <!-- not GET, not /submit -->
  <input type="range" name="level">  <!-- name = your dict key -->
  <button type="submit" name="btn" value="on">ON</button>
</form>
```

- **Always specify method and action**

  The default is GET aimed at a path you do not serve. Ask for method="POST" and action="/".

- **Name every input**

  The name attribute becomes the dictionary key your parser sees. An unnamed input sends nothing at all.

- **Strip what you cannot serve**

  External stylesheets, CDN scripts and favicon links will all 404 against your tiny server.

- **Verify against your parser**

  Paste the form in, click submit, and print the dict. The keys must match what your code looks for.

Serving HTTP: Things to Watch For

The blank line goes after the LAST header

\r\n\r\n once, separating headers from body. Send it after every header and the browser renders nothing.

Browsers also request /favicon.ico

An extra connection you did not ask for. Your loop will appear to fire twice per page load — that is why.

One client at a time

listen(3) only queues; a slow client still blocks the next.

The board must be reachable

Same subnet, client isolation off.

No sudo, no privileged ports

Unlike the Raspberry Pi, MicroPython has no OS enforcing port permissions. Port 80 is yours for the taking.

Reset if a socket sticks

A crashed script can leave port 80 bound. machine.reset() or ctrl-D from the Thonny REPL clears it.

03
requests

The board as a client

Turning the ESP32 Into a Client

Everything so far had your laptop calling the ESP32. Now the ESP32 calls out.

1

This direction works on umd-iot

Outbound connections are allowed on umd-iot, on eduroam, and behind almost any firewall.

2

`requests` is MicroPython's HTTP client

Same API shape as CPython's third-party requests library, trimmed to fit on a microcontroller.

3

If needed, install it with mip

Firmware generally ships urequests built in. If not, install with: mip.install('requests')

4

Outbound only

`requests` does not accept incoming connections. For that you still need a socket server, or a broker.

Making a GET Request

`requests.get()` replaces the entire socket + header dance with one call

```
import mip
mip.install('requests')      # once, from the REPL
```

```
import requests
import json

r = requests.get('https://api.github.com')

print(r.status_code)               # 200, 404, ...
print(r.headers['Content-Type'])
print(r.text)                      # body as text
data = r.json()                    # parsed into a dict

r.close()       # free the socket -- important here!
```

- **TLS comes for free**

  https:// works because MicroPython's ssl module handles the handshake — the same mechanism the MQTT broker used.

- **Check status_code first**

  An error page still returns an object. Calling .json() on a 404 body will raise.

- **Always close()**

  The board has very few sockets. Leaking them will make your next request fail for no visible reason.

- **Memory is the real limit**

  .text pulls the whole body into RAM. A large response will fail with a memory allocation error.

request GET example: weather_client.py

```
import wifi, requests, time

STATION = 'KCGS'                    # College Park Airport -- nearest station
URL = f'https://api.weather.gov/stations/{STATION}/observations/latest'

# api.weather.gov answers 403 to a request that does not identify itself:
HEADERS = {'User-Agent': '(ENME441 project, you@terpmail.umd.edu)'}

wifi.connect()

while True:
    r = requests.get(URL, headers=HEADERS)
    try:
        obs = r.json()['properties']    # parse 4.7 kB of JSON into a dict
    finally:
        r.close()                       # urequests will not free the socket
    temp_c   = obs['temperature']['value']       # degrees C, or None
    humidity = obs['relativeHumidity']['value']  # percent, or None
    print(f"{obs['timestamp']}  {temp_c} C  {humidity} %")
    time.sleep(600)                     # stations report about once an hour
```

- **User-Agent is not optional in this example**

  api.weather.gov answers HTTP 403 to a request that does not identify itself. The NWS asks for an application name and a contact address.

- **Always close the response**

  urequests does not release the socket on its own. Skip r.close() and the board runs out of sockets after a handful of requests.

- **Any field can be null**

  A station that missed its hourly report still returns valid JSON, with value set to None. Check before doing arithmetic on it.

Making a POST Request

Two body formats cover almost everything you will meet.

JSON — what most REST APIs expect

```
payload = json.dumps({'sensor': 21.4, 'unit': 'C'})

r = requests.post('https://httpbin.org/post',
        headers={'Content-Type': 'application/json'},
        data=payload)

print(r.status_code)      # 200 = OK, 201 = created
print(r.json())
r.close()
```

Form-encoded — same shape as an HTML form

```
r = requests.post(url,
        headers={'Content-Type':
                 'application/x-www-form-urlencoded'},
        data='val1=-2.2&val2=6&pin2=on')
```

- **data= becomes the body**

  Exactly the body you were slicing by hand two sections ago — now the library assembles it.

- **The header must match the body**

  Send JSON with a form-encoded Content-Type and the server will reject it, or silently misread it.

- **201 means created**

  A REST API that made a new resource answers 201, not 200. Both are success.

- **Same protocol, both directions**

  Your ESP32 server parsed these requests; now it writes them. One format, two roles.

request POST example: discord_client.py

```
import urequests as requests
import ujson
from machine import Pin

p_in = Pin(25, Pin.IN)

# Discord: channel -> gear icon -> Integrations -> Webhooks
url = 'https://discord.com/api/webhooks/<id>/<token>'

while True:
    data = ujson.dumps({'content' : str(p_in.value()),
                        'username': 'ENME441 bot'})
    r = requests.post(url,
            headers={'content-type': 'application/json'},
            data=data)
    print(r.text)
    r.close()
    sleep(10)
```

- **The URL is the credential**

  Anyone holding it can post to your channel.

- **'content' is the only required field**

  username optionally overrides the bot's display name. The rest of the payload is documented by Discord.

- **Same pattern everywhere**

  Slack, Teams, IFTTT, PagerDuty, etc. all expose the identical POST-a-JSON-body interface.

- **Mind the rate limit**

  Discord will start rejecting you well below one message per second. sleep(10) here is deliberate.

Data Security

Everything you have written so far is plain text that is readable by anyone on the network.

1

GET and POST send data in the clear

Your status lines, headers and form values all travel as ordinary text. A packet capture reads them without effort.

2

HTTPS is HTTP inside TLS

The protocol itself is unchanged. A TLS layer encrypts the bytes before they reach the wire and decrypts them at the far end.

3

Consuming HTTPS is easy

requests.get('https://...') already does it. MicroPython's ssl module handles certificates and the handshake for you.

4

Serving HTTPS from the board is difficult

It needs a certificate the browser will trust, plus RAM the ESP32 barely has. Keep your own server on a network you trust.

Summary

Aspect	Raw socket	HTTP server	requests client
Board's role	Either end	Server — it listens	Client — it calls out
Who connects	Whichever you write	Your laptop → board	Board → internet
Works on umd-iot	No — inbound blocked	No — inbound blocked	Yes — outbound is fine
Message format	Anything you define	HTTP + HTML	HTTP + JSON
Typical use	Device-to-device links	Local control panel	Cloud APIs, webhooks
Effort	Highest — you frame it all	Medium — you write headers	Lowest — one call

What to Remember

socket is the foundation

HTTP, MQTT and the web all run through the same bind / listen / accept / send / recv calls you saw in Part 1.

Bytes on the wire, always

encode() on the way out, decode() on the way in. Most socket bugs are exactly this and nothing more.

\r\n\r\n ends the headers

One blank line, sent once, after the last header. Get it wrong and the browser shows a blank page.

GET reads, POST writes

And POST data lives in the body — which is why you slice past the blank line to find it.

Serving needs a friendly network; calling out does not

Inbound connections are the fragile half. That asymmetry is the whole reason a broker exists.

Lab 11 – Peer to Peer Communication